SOFTWARE REQUIREMENTS SPECIFICATION

Hospital Patient Management System

হাসপাতাল রোগী ব্যবস্থাপনা সিস্টেম

Document Title	Hospital PMS — Software Requirements Specification (SRS)
Version	1.0
Date	May 2026
Standard	IEEE Std 830-1998
Prepared by	Dr. Engr. Nasir Ahmed
Course Name	AI Based Software Engineering
Batch No.	Soft 26/4
Email	ahnasir236@gmail.com
Project Name	Hospital Patient Management System (Hospital PMS)
Organization	ICT BGD, Dhaka, Bangladesh
Confidentiality	Academic / Internal Use Only

TABLE OF CONTENTS

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the Hospital Patient Management System (Hospital PMS). It is intended to serve as a reference for developers, testers, project managers, and hospital administrators involved in the development and deployment of the system.

This document follows the IEEE 830 standard for SRS documentation and covers all modules of the Hospital PMS including patient registration, appointment scheduling, billing, prescriptions, lab diagnostics, and reporting.

1.2 Scope

The Hospital Patient Management System (Hospital PMS) is a full-stack web-based application designed to digitize and streamline the daily operations of a multi-department hospital in Bangladesh. The system provides:

- A secure web interface accessible from any browser
- Role-based access for hospital staff (Admin, Doctor, Receptionist, Nurse, Lab Technician)
- Comprehensive patient lifecycle management from registration to discharge
- Integrated billing with Bangladesh-specific payment methods (bKash, Nagad, Rocket)
- Digital prescription writing and lab order management
- Real-time reports and analytics for hospital management

The system does NOT cover: mobile native applications, medical device integration, insurance claim processing, or payroll management in this version.

1.3 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
SRS	Software Requirements Specification
PMS	Patient Management System
API	Application Programming Interface
REST	Representational State Transfer
JWT	JSON Web Token — used for authentication
DRF	Django REST Framework
OPD	Outpatient Department
IPD	Inpatient Department
CRUD	Create, Read, Update, Delete operations

Term / Acronym	Definition
NID	National Identity Card (Bangladesh)
BMDC	Bangladesh Medical and Dental Council
bKash	Mobile banking service in Bangladesh (bKash Limited)
Nagad	Mobile financial service by Bangladesh Post Office
Rocket	Mobile banking service by Dutch-Bangla Bank
SSLCommerz	Payment gateway supporting Bangladesh mobile banking
UUID	Universally Unique Identifier
UI	User Interface
DB	Database
CORS	Cross-Origin Resource Sharing
DXA	Document eXtended Attribute — unit used in Word documents

1.4 References

1. IEEE 830-1998: IEEE Recommended Practice for Software Requirements Specifications
2. Django Documentation v6.0 — https://docs.djangoproject.com
3. React Documentation v19 — https://react.dev
4. PostgreSQL 15 Documentation — https://www.postgresql.org/docs
5. SSLCommerz Payment Gateway API — https://developer.sslcommerz.com
6. DGDA Bangladesh (Directorate General of Drug Administration) Medicine Guidelines
7. Bangladesh NID API Documentation — Election Commission Bangladesh

1.5 Overview

This SRS is organized into the following major sections:

- Section 1 — Introduction: purpose, scope, definitions, and references
- Section 2 — Overall Description: product perspective, functions, user classes, constraints
- Section 3 — Specific Requirements: detailed functional requirements for each module
- Section 4 — Non-Functional Requirements: performance, security, usability
- Section 5 — System Architecture: technology stack and deployment
- Section 6 — Database Requirements: tables, relationships, constraints
- Section 7 — External Interface Requirements: UI, APIs, hardware
- Section 8 — Appendix: use case summary, data flow

2. Overall Description

2.1 Product Perspective

The Hospital PMS is a new, self-contained web-based system. It replaces manual paper-based processes in the hospital. The system consists of two primary components:

- Backend REST API — built with Django 6.0.5 and Django REST Framework, running on port 8000
- Frontend Web Application — built with React 19 and Vite, running on port 5173

The system communicates via HTTP/HTTPS REST API calls. The frontend is a Single Page Application (SPA) that consumes the backend API using JWT authentication tokens.

2.2 Product Functions

The major functions of the Hospital PMS are summarized below:

Module	Primary Function
Authentication	Secure login/logout with JWT tokens and role-based access control
Patient Management	Register, view, update, search, and deactivate patient records
Appointment Scheduling	Book, manage, and track patient appointments by department and doctor
Billing & Payments	Generate invoices, collect payments via bKash/Nagad/Rocket/Cash/Card
Prescription Management	Write and view digital prescriptions with medicine details
Lab & Diagnostics	Order lab tests, track sample workflow, record results
Reports & Analytics	View revenue, patient counts, department statistics in real time
Admin Panel	Django admin for user, department, and system management

2.3 User Classes and Characteristics

User Role	Description	Key Permissions
Admin	Hospital IT administrator or manager	Full access to all modules, user management
Doctor	Licensed medical practitioner (BMDC registered)	Write prescriptions, order lab tests, view patient history
Receptionist	Front-desk hospital staff	Register patients, book appointments, create invoices
Nurse	Ward/OPD nursing staff	View patients and appointments, record vitals
Lab Technician	Laboratory staff	Manage lab orders, update test results and status

2.4 Operating Environment

Component	Requirement
Operating System	Windows 10/11, Linux Ubuntu 20+, macOS 12+
Backend Server	Python 3.12+, Django 6.0.5, Django REST Framework 3.17.1
Frontend Browser	Chrome 90+, Firefox 90+, Edge 90+, Safari 14+
Database	PostgreSQL 15+ with pgAdmin 4
Development IDE	PyCharm (backend), any editor with Node.js support (frontend)
Network	LAN or internet connection for API communication
Minimum RAM	8 GB (development), 4 GB (production server minimum)
Minimum Storage	10 GB free disk space for application and database

2.5 Design and Implementation Constraints

- The system must use Django REST Framework for all backend APIs
- All API endpoints must be protected with JWT authentication
- The frontend must be a React single-page application
- The database must be PostgreSQL — SQLite is not permitted in production
- Payment processing must use SSLCommerz for bKash, Nagad, and Rocket integration
- All dates and times must use the Asia/Dhaka timezone (UTC+6)
- Patient IDs must follow the format: P-YYYY-NNNN (e.g., P-2026-0001)
- Invoice numbers must follow the format: INV-YYYY-NNNN
- Lab order numbers must follow the format: LAB-YYYY-NNNN
- The system must support both Bengali and English text throughout the interface
- All sensitive configuration (passwords, API keys) must be stored in .env files

2.6 Assumptions and Dependencies

Assumptions

- Hospital has a stable internet connection or local area network
- All hospital staff have access to a modern web browser
- PostgreSQL 15 is installed and running before deployment
- SSLCommerz account is created and verified for payment processing
- SSL Wireless account is active for SMS gateway functionality

Dependencies

- Python 3.12+ must be installed on the server
- Node.js 20+ must be installed for building the frontend
- PostgreSQL must be running and accessible
- Internet connectivity required for bKash/Nagad payment processing

3. Specific Functional Requirements

3.1 Authentication Module

3.1.1 User Login

The system shall provide a secure login mechanism for all hospital staff.

Requirement ID	Requirement Description	Priority
AUTH-01	The system shall authenticate users with a username and password	High
AUTH-02	The system shall return a JWT access token (8-hour validity) upon successful login	High
AUTH-03	The system shall return a JWT refresh token (7-day validity) for token renewal	High
AUTH-04	The system shall display an error message for invalid credentials	High
AUTH-05	The system shall redirect authenticated users to the dashboard	High
AUTH-06	The system shall redirect unauthenticated users to the login page	High
AUTH-07	The system shall automatically attach the JWT token to all API requests	High
AUTH-08	The system shall redirect to login when a token expires (HTTP 401)	High

3.1.2 User Logout

Requirement ID	Requirement Description	Priority
AUTH-09	The system shall invalidate the JWT refresh token upon logout	High
AUTH-10	The system shall clear all locally stored tokens upon logout	High
AUTH-11	The system shall redirect the user to the login page after logout	High

3.1.3 Role-Based Access Control

Requirement ID	Requirement Description	Priority
AUTH-12	The system shall enforce role-based access: Admin, Doctor, Receptionist, Nurse, Lab Technician	High
AUTH-13	Only Admin users shall be able to create new user accounts	High
AUTH-14	Doctors shall only write prescriptions and order lab tests	Medium
AUTH-15	Receptionists shall manage patient registration, appointments, and billing	Medium

3.2 Patient Management Module

3.2.1 Patient Registration

Requirement ID	Requirement Description	Priority
PAT-01	The system shall allow registration of new patients with full personal details	High
PAT-02	The system shall auto-generate a unique Patient ID in format P-YYYY-NNNN	High
PAT-03	Patient ID shall be unique, indexed, and non-editable after creation	High
PAT-04	The system shall capture: first name, last name, date of birth, gender, blood group, NID, phone, emergency contact, address, district, allergies, chronic conditions	High
PAT-05	The system shall validate that required fields (name, DOB, phone, address, district) are not empty	High
PAT-06	The system shall support Bangladesh districts in a dropdown selector	Medium
PAT-07	The system shall display a success message after successful registration	Medium
PAT-08	The system shall auto-calculate patient age from date of birth	Low

3.2.2 Patient Search and Listing

Requirement ID	Requirement Description	Priority
PAT-09	The system shall display all active patients in a paginated table	High
PAT-10	The system shall support real-time search by name, phone, patient ID, and NID	High
PAT-11	The system shall show: patient ID, full name, age, gender, blood group, phone, district, status	High
PAT-12	The system shall support soft-delete (deactivation) of patients — not hard delete	High
PAT-13	The system shall show a total patient count at the bottom of the list	Low

3.3 Appointment Management Module

3.3.1 Department Management

Requirement ID	Requirement Description	Priority
APT-01	The system shall maintain a list of hospital departments (Medicine, Surgery, Cardiology, etc.)	High
APT-02	Each department shall have a unique code (e.g., MED, CARD, GYN)	High
APT-03	Departments shall support active/inactive status	Medium

3.3.2 Doctor Management

Requirement ID	Requirement Description	Priority
APT-04	Each doctor shall be linked to a system User account with role DOCTOR	High
APT-05	Doctor profile shall include: specialization, qualification, BMDC license number, consultation fee, available days	High
APT-06	Doctors shall be assigned to a department	High
APT-07	The system shall filter doctors by department	Medium

3.3.3 Appointment Booking

Requirement ID	Requirement Description	Priority
APT-08	The system shall allow booking of appointments for registered patients	High
APT-09	An appointment shall require: patient, doctor, department, date/time, type (OPD/Follow-up/Emergency)	High
APT-10	Appointment status shall flow: SCHEDULED → CONFIRMED → IN_PROGRESS → COMPLETED	High
APT-11	The system shall support cancellation and no-show status	Medium
APT-12	The system shall display today's appointments with a dedicated view	High
APT-13	The system shall allow filtering appointments by date and status	Medium
APT-14	The system shall display patient name and doctor name in the appointment list	High

3.4 Billing and Payment Module

3.4.1 Invoice Management

Requirement ID	Requirement Description	Priority
BIL-01	The system shall generate invoices for hospital services	High
BIL-02	Invoice numbers shall be auto-generated in format INV-YYYY-NNNN	High
BIL-03	Each invoice shall support multiple line items with description, quantity, and unit price	High
BIL-04	The system shall auto-calculate subtotal, discount, tax, and total	High
BIL-05	Invoice status shall be: PENDING, PAID, PARTIAL, CANCELLED	High
BIL-06	The system shall display total collected revenue and pending amounts	High

3.4.2 Payment Collection

Requirement ID	Requirement Description	Priority
BIL-07	The system shall support payment via: Cash, bKash, Nagad, Rocket, Credit/Debit Card	High
BIL-08	For mobile payments (bKash, Nagad, Rocket), the system shall accept a transaction ID	High
BIL-09	Upon full payment, invoice status shall automatically change to PAID	High
BIL-10	Upon partial payment, invoice status shall change to PARTIAL	High
BIL-11	The system shall record the payment date and time	High
BIL-12	Invoices shall be filterable by status (Paid, Pending, Partial, Cancelled)	Medium

3.4.3 Invoice Printing

Requirement ID	Requirement Description	Priority
BIL-13	The system shall generate a printable invoice in HTML/PDF format	High
BIL-14	The printed invoice shall display: hospital name, invoice number, patient name, date, line items, totals, payment details	High
BIL-15	The invoice shall include Bengali text (হাসপাতাল ব্যবস্থাপনা সিস্টেম)	Medium
BIL-16	The invoice print dialog shall open automatically when Print is clicked	Medium
BIL-17	Users shall be able to save the invoice as a PDF file	Medium

3.5 Prescription Management Module

Requirement ID	Requirement Description	Priority
RX-01	Doctors shall be able to write digital prescriptions for patients	High
RX-02	Each prescription shall include: diagnosis, advice, follow-up date, and a list of medicines	High
RX-03	Each medicine item shall include: medicine name, dosage, frequency, duration, instructions	High
RX-04	Medicine frequency options shall be: OD (once daily), BD (twice daily), TDS, QDS, SOS, STAT	High
RX-05	The system shall support multiple medicines per prescription	High
RX-06	Prescriptions shall be viewable in a detail modal showing all medicine information	High
RX-07	The system shall display a count of medicines per prescription in the list view	Low
RX-08	Prescriptions shall be linked to the prescribing doctor and the patient	High

3.6 Lab and Diagnostics Module

3.6.1 Lab Test Catalogue

Requirement ID	Requirement Description	Priority
LAB-01	The system shall maintain a catalogue of available lab tests	High
LAB-02	Each lab test shall have: name, code, description, price, turnaround time in hours	High
LAB-03	Lab tests shall support active/inactive status	Medium

3.6.2 Lab Order Management

Requirement ID	Requirement Description	Priority
LAB-04	Doctors shall be able to order lab tests for patients	High
LAB-05	Lab order numbers shall auto-generate in format LAB-YYYY-NNNN	High
LAB-06	Each lab order shall support multiple tests	High
LAB-07	Lab order status shall flow: ORDERED → SAMPLE_COLLECTED → IN_PROGRESS → COMPLETED	High
LAB-08	Lab Technicians shall be able to update the status of each order	High
LAB-09	The system shall record completion date and time when status is set to COMPLETED	Medium
LAB-10	Test results shall be recordable per test item within an order	High
LAB-11	The system shall flag test results as normal or abnormal	Medium
LAB-12	Orders shall be filterable by status	Medium

3.7 Reports and Analytics Module

Requirement ID	Requirement Description	Priority
RPT-01	The system shall display total patient count on the reports page	High
RPT-02	The system shall display total appointment count	High
RPT-03	The system shall display total revenue collected from paid invoices	High
RPT-04	The system shall display total pending payment amount	High
RPT-05	The system shall display total invoice count, prescription count, lab order count	High
RPT-06	The system shall show a list of recent patients with patient ID, district, and blood group	Medium
RPT-07	The system shall show recent invoices with status and amounts	Medium
RPT-08	The system shall display department overview cards with active/inactive status	Medium
RPT-09	The reports page shall include a Refresh button to reload live data	Medium

3.8 Admin Panel Requirements

Requirement ID	Requirement Description	Priority
ADM-01	The system shall provide a Django Admin panel at /admin	High
ADM-02	Admin shall manage users with role assignment	High
ADM-03	Admin shall manage departments, doctors, and appointments	High
ADM-04	Admin shall view and edit invoices with inline line items	High
ADM-05	Admin shall manage lab tests catalogue	Medium
ADM-06	Admin shall view prescriptions with inline medicine items	Medium
ADM-07	Admin panel shall support search and filtering for all models	Medium

4. Non-Functional Requirements

4.1 Performance Requirements

Requirement ID	Requirement Description	Target
PERF-01	API response time for patient list (up to 100 records)	< 500ms
PERF-02	Login authentication response time	< 1 second
PERF-03	Invoice creation with 5 line items	< 1 second
PERF-04	Dashboard load time (initial)	< 2 seconds
PERF-05	Concurrent users supported (development environment)	10–20 users
PERF-06	Database query optimization via indexed fields (patient_id, nid_number)	Required

4.2 Security Requirements

Requirement ID	Requirement Description	Priority
SEC-01	All API endpoints (except /api/auth/login/) shall require JWT Bearer token authentication	Critical
SEC-02	JWT access tokens shall expire after 8 hours	Critical
SEC-03	JWT refresh tokens shall expire after 7 days	Critical
SEC-04	Passwords shall be stored as hashed values using Django's PBKDF2-SHA256	Critical
SEC-05	The system shall implement CORS protection allowing only the React frontend origin	High
SEC-06	All sensitive configuration (DB password, secret keys, payment API keys) shall be stored in .env files and never committed to version control	Critical
SEC-07	The Django SECRET_KEY shall be a cryptographically strong random string in production	Critical
SEC-08	DEBUG mode shall be set to False in production deployment	Critical
SEC-09	The system shall validate all user input on both frontend and backend	High
SEC-10	SQL injection shall be prevented through Django's ORM parameterized queries	Critical

4.3 Usability Requirements

Requirement ID	Requirement Description	Priority
USE-01	The interface shall be navigable from a left sidebar with icons and labels	High
USE-02	The active menu item shall be visually highlighted in the sidebar	Medium
USE-03	All forms shall display validation error messages inline	High
USE-04	Success and error messages shall be displayed as colored notification banners	High
USE-05	The interface shall support both Bengali and English text	High
USE-06	Bangladesh-specific dropdown fields shall include all 64 districts	Medium
USE-07	The application shall be responsive for desktop screens (minimum 1024px width)	High
USE-08	Loading states shall be shown for all asynchronous operations	Medium

4.4 Reliability Requirements

- The system shall be available during all hospital working hours (minimum 99% uptime in production)
- Database backups shall be performed daily in production environments
- The system shall handle API errors gracefully and display user-friendly error messages
- Network timeout errors shall be caught and displayed to the user without crashing the application
- Patient data shall never be permanently deleted — only soft-deleted (is_active = False)

4.5 Maintainability Requirements

- All Django apps shall follow the standard structure: models.py, serializers.py, views.py, urls.py, admin.py
- All API endpoints shall follow RESTful naming conventions
- Environment-specific settings shall be separated using .env files and python-decouple
- The codebase shall be structured to allow adding new modules without modifying existing ones
- Database migrations shall be version-controlled alongside the codebase

4.6 Portability Requirements

- The backend shall run on any operating system supporting Python 3.12+ (Windows, Linux, macOS)
- The frontend shall be compatible with Chrome 90+, Firefox 90+, Edge 90+, and Safari 14+
- The system shall be deployable on cloud platforms (DigitalOcean, AWS, Heroku, or equivalent)
- Database migrations shall ensure portability between development and production PostgreSQL instances

5. System Architecture

5.1 Architecture Overview

The Hospital PMS follows a three-tier architecture:

Tier	Component	Technology	Port
Presentation Tier	React Frontend (SPA)	React 19 + Vite + Tailwind CSS v4	5173
Application Tier	Django REST API	Django 6.0.5 + Django REST Framework 3.17.1	8000
Data Tier	Relational Database	PostgreSQL 15 (managed via pgAdmin 4)	5432

5.2 Technology Stack

5.2.1 Backend Stack

Package	Version	Purpose
Django	6.0.5	Web framework — URL routing, ORM, admin panel
djangorestframework	3.17.1	REST API views, serializers, authentication
djangorestframework-simplejwt	5.5.1	JWT token generation, validation, refresh
django-cors-headers	4.9.0	Cross-Origin Resource Sharing for React frontend
django-filter	25.2	Queryset filtering for API endpoints
psycopg2-binary	2.9.12	PostgreSQL database adapter for Python
python-decouple	3.8	Environment variable management from .env file

5.2.2 Frontend Stack

Package	Version	Purpose
React	19+	UI component library
Vite	8+	Frontend build tool and development server
Tailwind CSS	v4	Utility-first CSS framework
@tailwindcss/vite	v4	Tailwind v4 Vite plugin
React Router DOM	7+	Client-side routing and navigation
@reduxjs/toolkit	2+	Redux state management with simplified API
react-redux	9+	React bindings for Redux
axios	1+	Promise-based HTTP client for API calls

5.3 Communication Flow

The system communicates as follows:

1. User opens browser and navigates to http://localhost:5173
2. React app loads; if no JWT token in localStorage, redirect to /login
3. User submits credentials; React POST to http://127.0.0.1:8000/api/auth/login/
4. Django validates credentials, returns JWT access + refresh tokens
5. React stores tokens in localStorage; redirects to dashboard
6. All subsequent API calls include Authorization: Bearer <access_token> header
7. If access token expires (401), Axios interceptor redirects to /login
8. React renders data received from API responses

6. Database Requirements

6.1 Database Overview

Database Engine: PostgreSQL 15
Database Name: hospital_pms
Connection: localhost:5432
Management Tool: pgAdmin 4

6.2 Database Tables

Table Name	Django App	Records	Description
authentication_user	authentication	Staff accounts	Custom user model extending AbstractUser with role field
patients_patient	patients	Patient records	Core patient data with auto-generated patient_id
appointments_department	appointments	Departments	Hospital departments (Medicine, Surgery, etc.)
appointments_doctor	appointments	Doctor profiles	Doctor details linked to User accounts
appointments_appointment	appointments	Appointments	Patient-doctor appointment records
billing_invoice	billing	Invoices	Invoice headers with auto-generated invoice numbers
billing_invoiceitem	billing	Invoice lines	Individual line items belonging to invoices
prescriptions_prescription	prescriptions	Prescriptions	Prescription headers with diagnosis and advice
prescriptions_prescriptionitem	prescriptions	Medicines	Individual medicine items in prescriptions
lab_labtest	lab	Test catalogue	Available lab tests with prices
lab_laborder	lab	Lab orders	Lab test orders with auto-generated order numbers
lab_laborderitem	lab	Order items	Individual tests within a lab order with results
django_migrations	django	Migrations	Applied migration history
django_session	django	Sessions	Admin panel session data
auth_group	django	Groups	Django permission groups

6.3 Key Data Constraints

- All primary keys use UUID (universally unique identifier) — not auto-increment integers
- patient_id (e.g., P-2026-0001) is unique, indexed, and auto-generated on save
- invoice_number (e.g., INV-2026-0001) is unique and auto-generated on save
- order_number (e.g., LAB-2026-0001) is unique and auto-generated on save
- Patient deletion is soft-delete only — is_active flag is set to False
- All timestamps use Asia/Dhaka timezone (UTC+6)
- Foreign key relationships use PROTECT (not CASCADE) for patient-related records to prevent accidental data loss

7. External Interface Requirements

7.1 User Interface Requirements

- The UI shall use a dark blue (#1E3A5F) sidebar for navigation
- The main content area shall use a light gray (#F3F4F6) background
- All tables shall use alternating row colors for readability
- Status badges shall use color coding: green = active/paid, yellow = pending, red = cancelled, blue = partial
- All monetary values shall display the Bengali Taka symbol (৳) before the amount
- The login page shall display both English title and Bengali subtitle
- Forms shall use blue focus rings on active input fields
- Primary action buttons shall use blue (#2563EB) background
- Destructive action buttons (logout, cancel) shall use red background

7.2 API Interface Requirements

All API endpoints follow RESTful conventions:

Convention	Requirement
Base URL	http://127.0.0.1:8000/api/ (development)
Authentication Header	Authorization: Bearer <JWT access token>
Content Type	application/json for all requests and responses
HTTP Methods	GET (read), POST (create), PUT (update), DELETE (soft-delete)
Success Response	HTTP 200 OK (read/update), HTTP 201 Created (create)
Error Response	HTTP 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found
Pagination	GET list endpoints return { count: N, results: [...] }
UUID in URLs	Resource IDs in URLs use UUID format: /api/patients/<uuid:pk>/

7.3 Payment Gateway Interface — SSLCommerz

Interface Item	Specification
Gateway	SSLCommerz — supports bKash, Nagad, Rocket, VISA, Mastercard
Sandbox URL	https://sandbox.sslcommerz.com/gwprocess/v4/api.php
Live URL	https://securepay.sslcommerz.com/gwprocess/v4/api.php
Configuration	SSLCOMMERZ_STORE_ID and SSLCOMMERZ_STORE_PASS in .env file
Currency	BDT (Bangladeshi Taka)
Callback URLs	success_url, fail_url, cancel_url configured per transaction
Transaction ID	Invoice number used as unique transaction reference

7.4 SMS Gateway Interface — SSL Wireless Bangladesh

Interface Item	Specification
Provider	SSL Wireless — Bangladesh SMS gateway
API URL	https://sms.sslwireless.com/pushapi/dynamic/server.php
Configuration	SMS_API_KEY and SMS_SENDER_ID in .env file
Use Case	Appointment reminders sent to patients
Message Format	English: Dear {name}, appointment with Dr. {doctor} on {date}

8. Appendix

8.1 Use Case Summary

Use Case ID	Use Case Name	Actor	Brief Description
UC-01	Login to System	All users	User enters credentials and receives JWT token
UC-02	Register Patient	Receptionist, Admin	Create new patient record with auto Patient ID
UC-03	Search Patient	All users	Find patient by name, phone, ID, or NID
UC-04	Book Appointment	Receptionist, Admin	Schedule appointment for patient with a doctor
UC-05	View Today's Appointments	Doctor, Receptionist	See all appointments scheduled for today
UC-06	Write Prescription	Doctor	Create digital prescription with medicines
UC-07	Order Lab Tests	Doctor	Order one or more lab tests for a patient
UC-08	Update Lab Order Status	Lab Technician	Move lab order through the workflow stages
UC-09	Create Invoice	Receptionist, Admin	Generate bill for hospital services
UC-10	Collect Payment	Receptionist, Admin	Record payment via Cash/bKash/Nagad/Rocket/Card
UC-11	Print Invoice	Receptionist, Admin	Generate printable PDF invoice
UC-12	View Reports	Admin, Manager	See revenue, patient stats, department overview
UC-13	Manage Users	Admin	Create, edit, and assign roles to system users
UC-14	Manage Departments	Admin	Create and manage hospital departments
UC-15	Logout	All users	Securely log out and invalidate JWT tokens

8.2 Data Flow Summary

Flow	Source	Process	Destination
Patient registration	Receptionist (UI form)	API POST /api/patients/	patients_patient table
JWT authentication	Login form credentials	Django authenticate() + SimpleJWT	localStorage (browser)
Appointment booking	Appointment form	API POST /api/appointments/	appointments_appointment table
Invoice creation	Billing form with items	API POST /api/billing/	billing_invoice + billing_invoiceitem tables
Payment collection	Payment modal	API POST /api/billing/<id>/pay/	billing_invoice (status update)
Prescription writing	Prescription form	API POST /api/prescriptions/	prescriptions_prescription + prescriptionitem tables
Lab order creation	Lab order form	API POST /api/lab/orders/	lab_laborder + lab_laborderitem tables
Reports data	Reports page load	Multiple GET API calls in parallel	React state (UI display)